Moveo Platform — MCA Final Year Major Project Blueprint

Comprehensive Software Engineering Diagram & Documentation Guide

> **Project Title:** Moveo — Enterprise Distributed Multi-Modal Travel & Ticket Booking Platform
> **Course / Degree:** Master of Computer Applications (MCA) — 1-Year Final Major Project
> **Architecture Stack:** Next.js (Frontend), Python FastAPI (10 Microservices), Apache Kafka (Event Bus), Redis (Distributed Lock & Cache), PostgreSQL (Database-per-Service), Elasticsearch (Search), PyTorch/Scikit-Learn (AI Engine), Docker Compose / Kubernetes (Deployment).

Table of Contents

1. Academic vs. Industry Expectations for MCA Major Projects
2. Master Diagram Checklist & Report Placement
3. Section A: Classical Data Flow Diagrams (DFD)
    - DFD Level 0 (Context Diagram)
    - DFD Level 1 (Functional System Decomposition)
    - DFD Level 2 (Booking & Saga Engine Decomposition)
4. Section B: Object-Oriented & Structural UML Diagrams
    - Unified Use Case Diagram
    - Detailed Class Diagram
    - Deployment & Physical Component Diagram
5. Section C: Behavioral & Interaction UML Diagrams
    - Sequence Diagram 1: User Authentication & JWT Flow
    - Sequence Diagram 2: Saga Distributed Booking & Compensation
    - Sequence Diagram 3: Transactional Outbox & Event Publishing
    - Master Activity Diagram (End-to-End User Booking Journey)
    - State Transition Diagram (Booking Lifecycle)
6. Section D: Database Schema & Entity Modeling
    - Database-per-Service ER Diagram
7. Section E: Viva Voce & Defense Strategy

---

1. Academic vs. Industry Expectations for MCA Major Projects

In an MCA 1-Year Major Project, university evaluators look for **two key dimensions**:

1. **Standard Academic Compliance**: DFDs (Level 0, 1, 2), UML Use Case, ERD, Class Diagram, Sequence Diagram, Activity Diagram, and Deployment Diagram.
2. **Advanced Architectural Distinction**: Standard college projects are simple monolithic CRUD applications. Moveo stands out because it implements **Microservices**, **Saga Pattern for Distributed Transactions**, **Transactional Outbox**, **Kafka Event-Driven Architecture**, and **AI-Assisted Fare Prediction**. Showing diagrams of these patterns will guarantee top marks.

---

2. Master Diagram Checklist & Report Placement

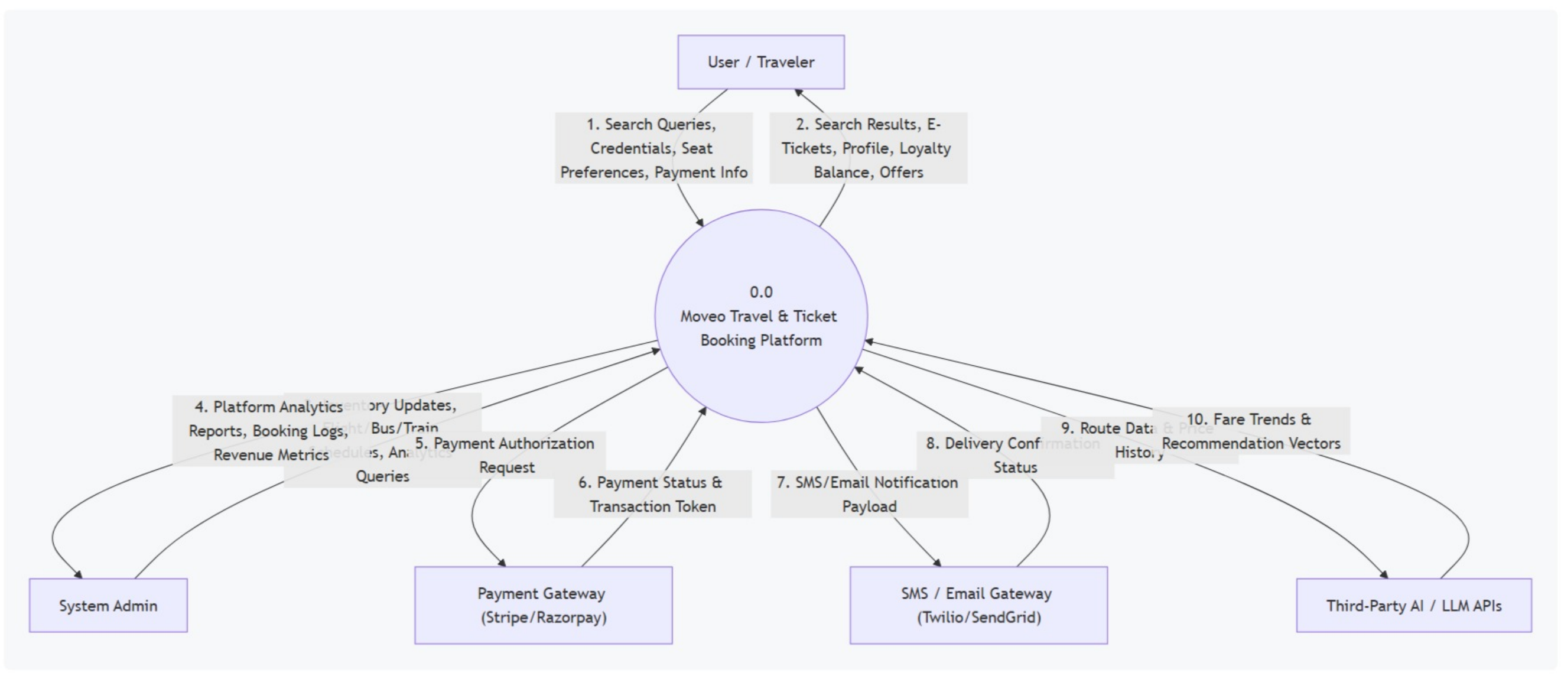
Diagram Name	Type	Project Report Chapter / Section	Essential for Synopsis?	Essential for Final Viva?
DFD Level 0 (Context)	Structured Analysis	Chapter 3: System Requirements & DFDs	Yes	Yes
DFD Level 1	Structured Analysis	Chapter 3: System Requirements & DFDs	Yes	Yes
DFD Level 2 (Booking Core)	Structured Analysis	Chapter 3: System Requirements & DFDs	Optional	Yes
Use Case Diagram	UML Behavioral	Chapter 4: System Design Document (SDD)	Yes	Yes
High-Level System Architecture	Architectural	Chapter 4: System Design Document (SDD)	Yes	Yes
ER Diagram (ERD)	Data Modeling	Chapter 4: Database Design	Yes	Yes
Class Diagram	UML Structural	Chapter 4: Detailed Object Design	Optional	Yes
Sequence Diagram (Saga)	UML Behavioral	Chapter 4/5: Process Interventions & Workflows	Optional	Yes
Activity Diagram	UML Behavioral	Chapter 4: Workflow Modeling	Yes	Yes
State Machine Diagram	UML Behavioral	Chapter 4: State Analysis	Optional	Yes
Deployment Diagram	UML Structural	Chapter 5: Deployment & Environment	Optional	Yes

---

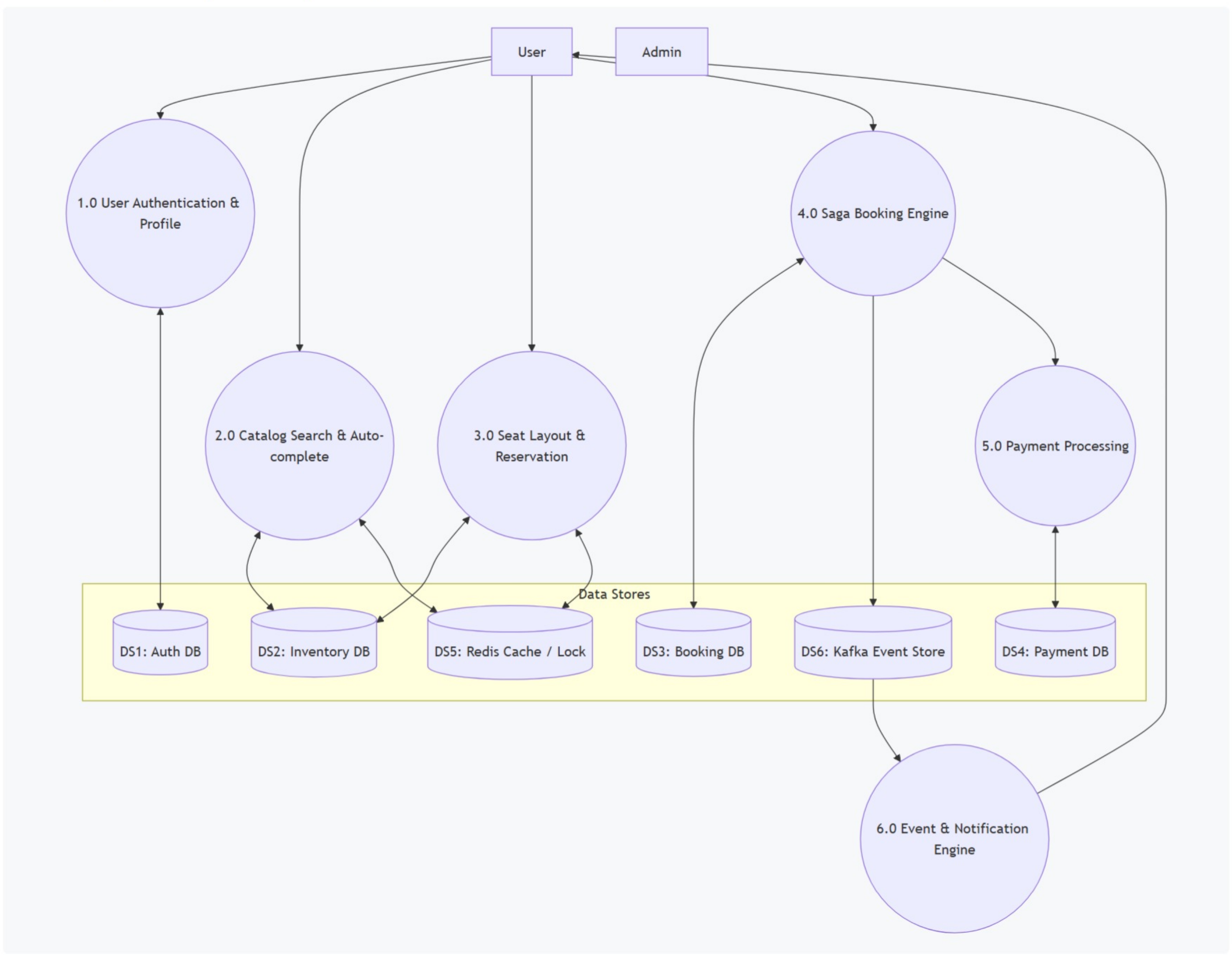
3. Section A: Classical Data Flow Diagrams (DFD)

DFD Level 0 (Context Diagram)

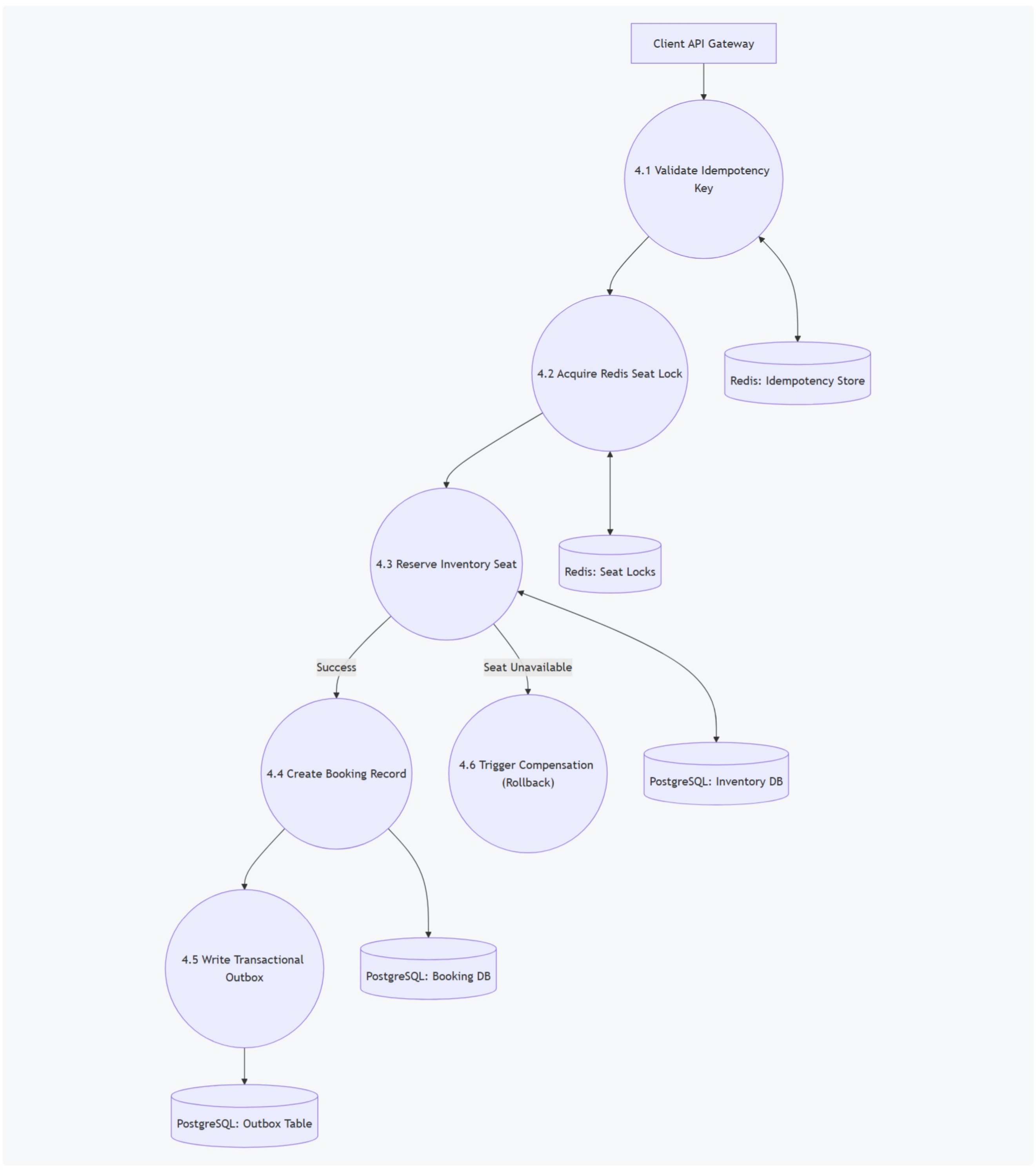
The Context Diagram establishes the system boundary for Moveo and its external entities.


User / Traveler
1. Search Queries, Credentials, Seat Preferences, Payment Info
2. Search Results, E-Tickets, Profile, Loyalty Balance, Offers
0.0
Moveo Travel & Ticket Booking Platform
4. Platform Analytics Reports, Booking Logs, Revenue Metrics
5. Payment Authorization Request
6. Payment Status & Transaction Token
7. SMS/Email Notification Payload
8. Delivery Confirmation Status
9. Route Data
10. Fare Trends & Recommendation Vectors
System Admin
Payment Gateway (Stripe/Razorpay)
SMS / Email Gateway (Twilio/SendGrid)
Third-Party AI / LLM APIs

DFD Level 1 (Functional System Decomposition)



DFD Level 2 (Booking & Saga Engine Decomposition)


Client API Gateway
4.1 Validate Idempotency Key
Redis: Idempotency Store
4.2 Acquire Redis Seat Lock
Redis: Seat Locks
4.3 Reserve Inventory Seat
Success
Seat Unavailable
PostgreSQL: Inventory DB
4.4 Create Booking Record
4.6 Trigger Compensation (Rollback)
4.5 Write Transactional Outbox
PostgreSQL: Booking DB
PostgreSQL: Outbox Table

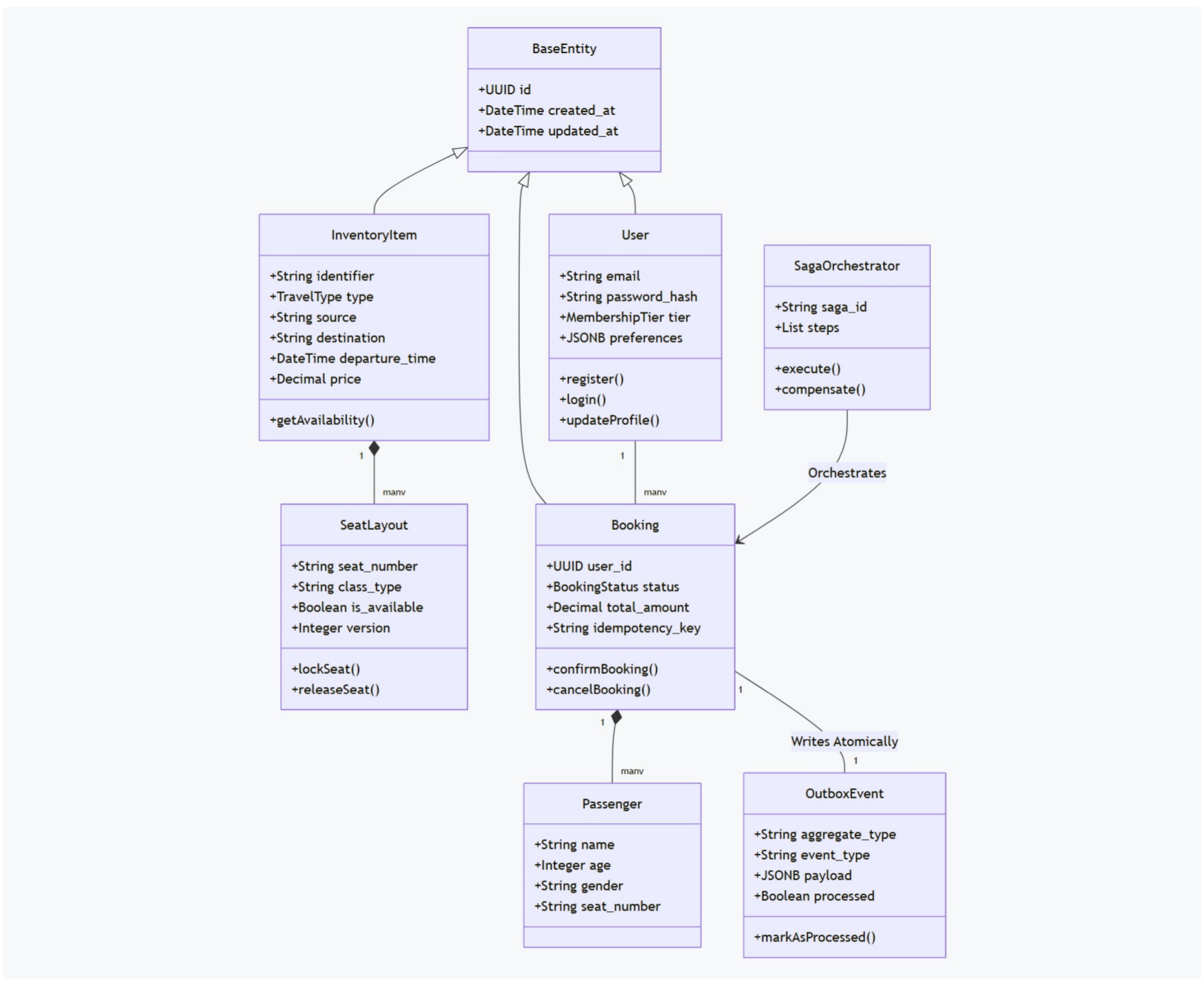
---

4. Section B: Object-Oriented & Structural UML Diagrams

Unified Use Case Diagram

```
Mermaid render error: Parse error on line 2:
graph LR    actor User as "Traveler /
------------------^
Expecting 'SEMI', 'NEWLINE', 'EOF', 'AMP', 'START_LINK', 'LINK', 'LINK_ID', got 'NODE_STRING'
```

Detailed Class Diagram



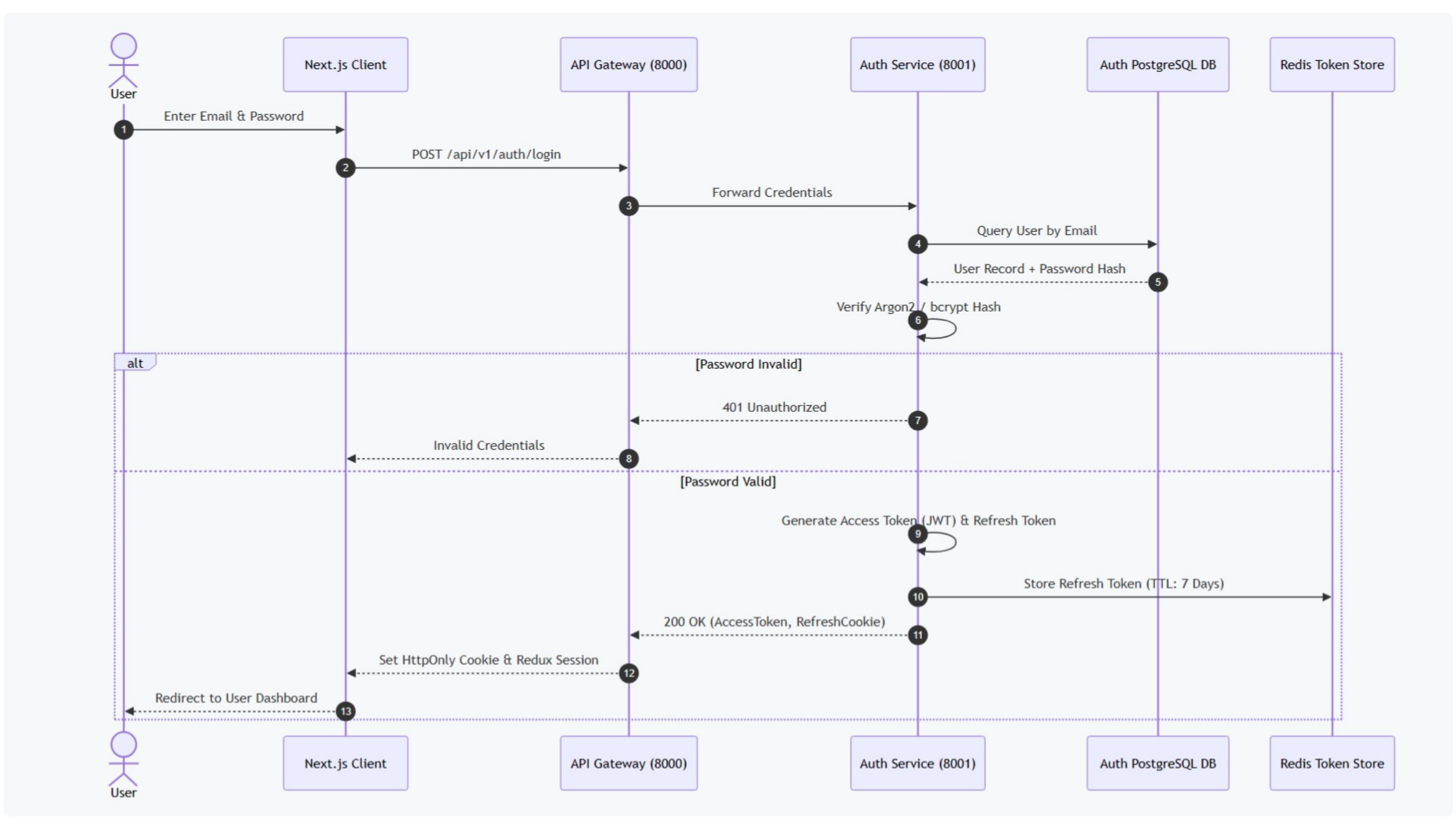
Deployment & Physical Component Diagram

```
Mermaid render error: Parse error on line 6:
...ess & Gateway Layer (Docker Network: pub
-----------------------^
Expecting 'SEMI', 'NEWLINE', 'SPACE', 'EOF', 'GRAPH', 'DIR', 'subgraph', 'SQS', 'end', 'AMP', 'COLON', 'START_LINK', 'STYLE', 'LINKSTYLE', 'CLASSDEF', 'CLASS', 'CLICK', 'DOWN', 'UP', 'NUM', 'NODE_STRING', 'BRKT', 'MINUS', 'MULT', 'UNICODE_TEXT', got 'PS'
```

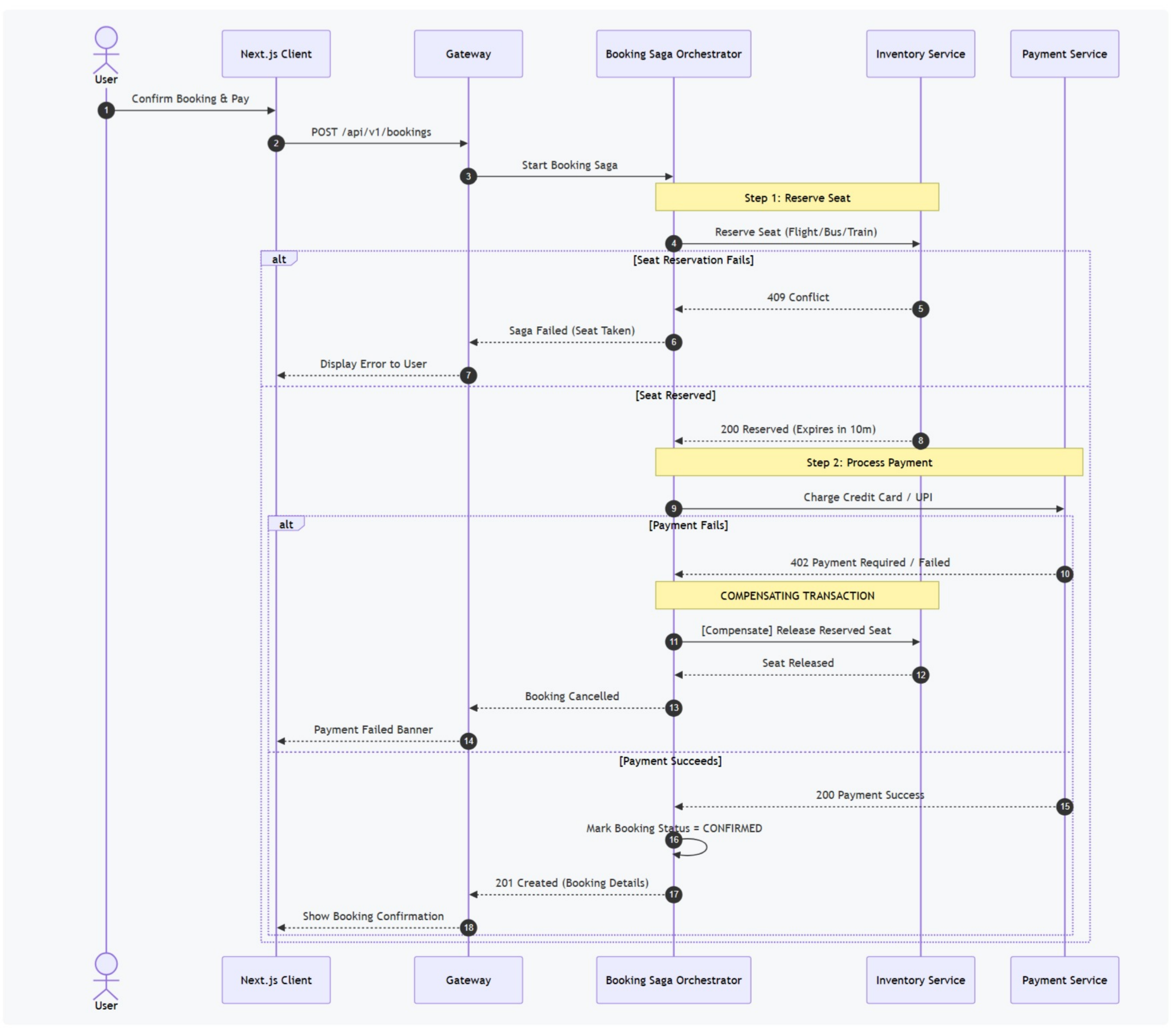
---

5. Section C: Behavioral & Interaction UML Diagrams

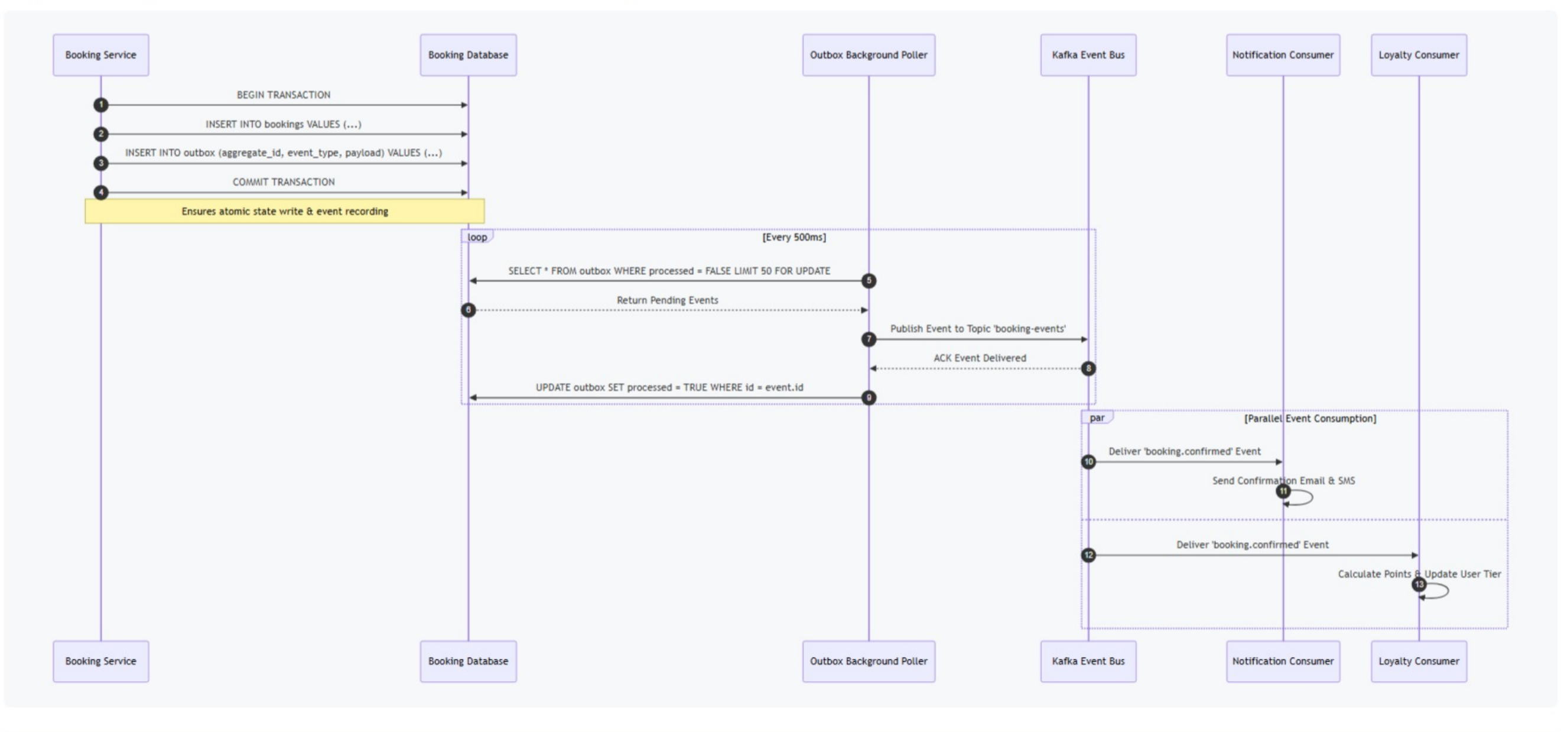
Sequence Diagram 1: User Authentication & JWT Flow


User
Next.js Client
API Gateway (8000)
Auth Service (8001)
Auth PostgreSQL DB
Redis Token Store
1 Enter Email & Password
2 POST /api/v1/auth/login
3 Forward Credentials
4 Query User by Email
5 User Record + Password Hash
6 Verify Argon2 / bcrypt Hash
alt
[Password Invalid]
7 401 Unauthorized
8 Invalid Credentials
[Password Valid]
9 Generate Access Token (JWT) & Refresh Token
10 Store Refresh Token (TTL: 7 Days)
11 200 OK (AccessToken, RefreshCookie)
12 Set HttpOnly Cookie & Redux Session
13 Redirect to User Dashboard

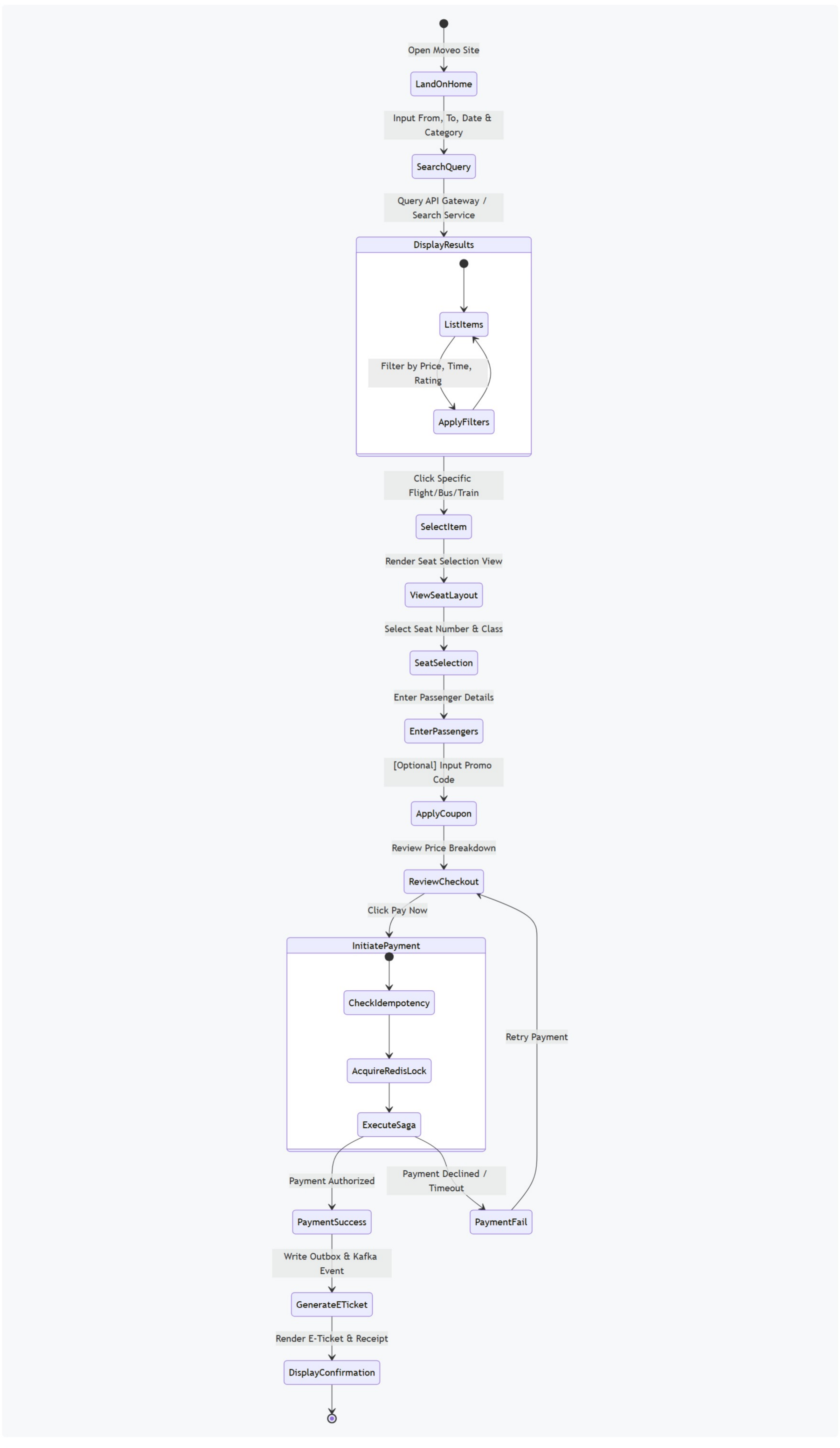
Sequence Diagram 2: Saga Distributed Booking & Compensation



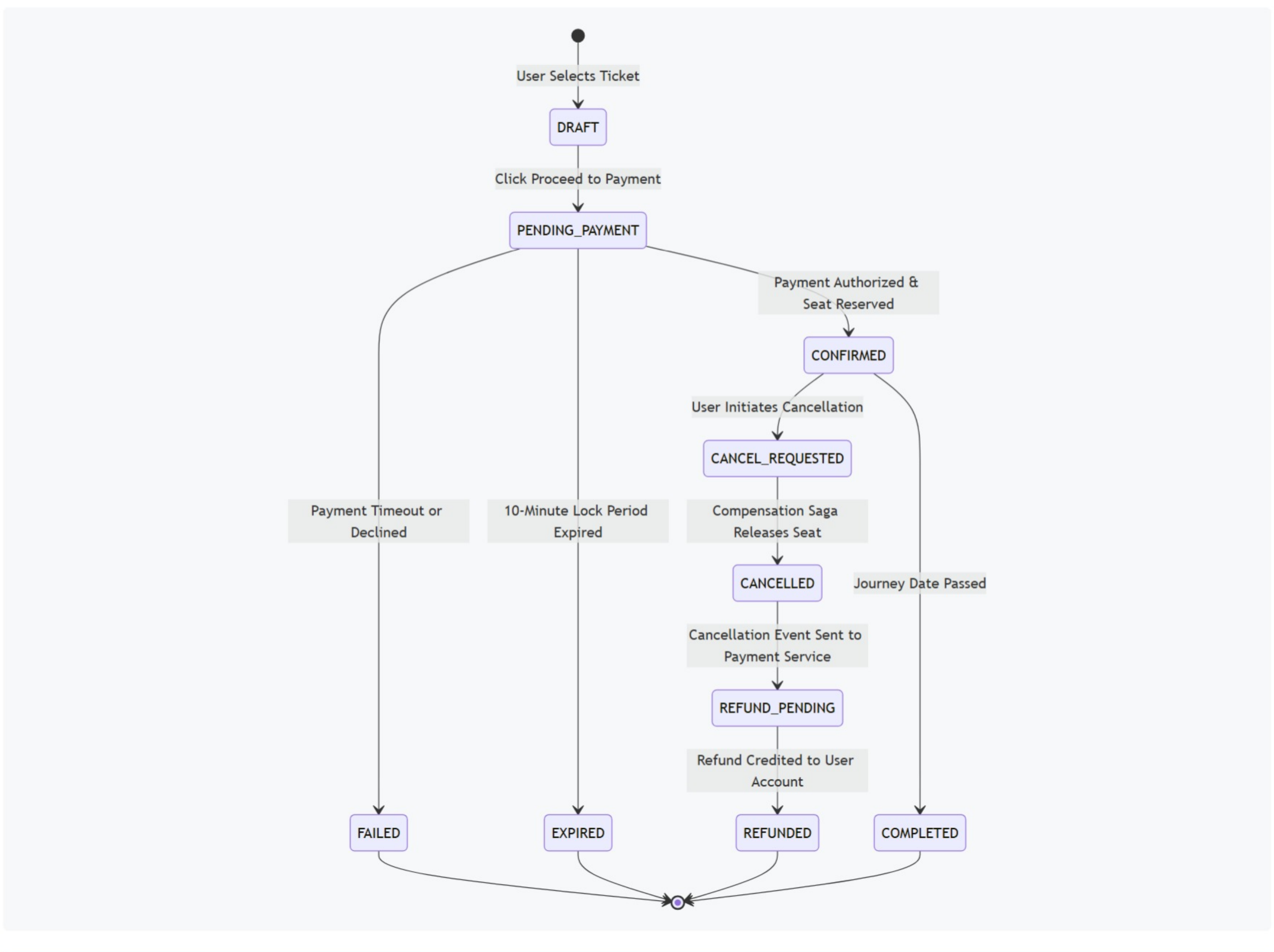
Sequence Diagram 3: Transactional Outbox & Event Publishing


Booking Service
Booking Database
Outbox Background Poller
Kafka Event Bus
Notification Consumer
Loyalty Consumer
1 BEGIN TRANSACTION
2 INSERT INTO bookings VALUES (...)
3 INSERT INTO outbox (aggregate_id, event_type, payload) VALUES (...)
4 COMMIT TRANSACTION
Ensures atomic state write & event recording
loop
[Every 500ms]
5 SELECT * FROM outbox WHERE processed = FALSE LIMIT 50 FOR UPDATE
6 Return Pending Events
7 Publish Event to Topic 'booking-events'
8 ACK Event Delivered
9 UPDATE outbox SET processed = TRUE WHERE id = event.id
par
[Parallel Event Consumption]
10 Deliver 'booking.confirmed' Event
11 Send Confirmation Email & SMS
12 Deliver 'booking.confirmed' Event
13 Calculate Points & Update User Tier

Master Activity Diagram (End-to-End User Booking Journey)



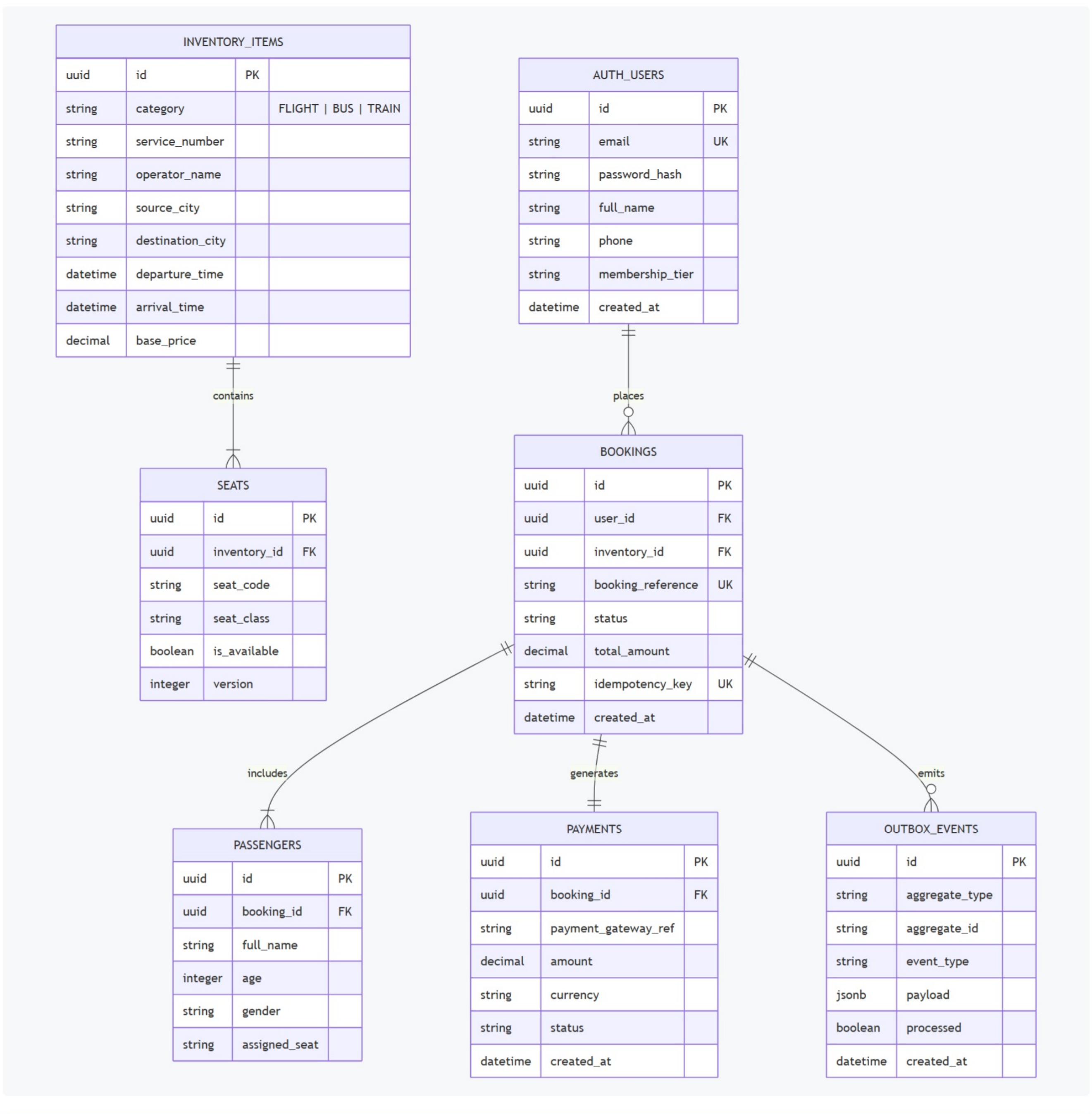
State Transition Diagram (Booking Lifecycle)


User Selects Ticket
DRAFT
Click Proceed to Payment
PENDING_PAYMENT
Payment Authorized & Seat Reserved
CONFIRMED
User Initiates Cancellation
CANCEL_REQUESTED
Compensation Saga Releases Seat
CANCELLED
Cancellation Event Sent to Payment Service
REFUND_PENDING
Refund Credited to User Account
Payment Timeout or Declined
10-Minute Lock Period Expired
Journey Date Passed
FAILED
EXPIRED
REFUNDED
COMPLETED

---

6. Section D: Database Schema & Entity Modeling

Database-per-Service ER Diagram



INVENTORY_ITEMS

uuid	id	PK	
string	category		FLIGHT | BUS | TRAIN
string	service_number		
string	operator_name		
string	source_city		
string	destination_city		
datetime	departure_time		
datetime	arrival_time		
decimal	base_price		

AUTH_USERS

uuid	id	PK
string	email	UK
string	password_hash	
string	full_name	
string	phone	
string	membership_tier	
datetime	created_at	

SEATS

uuid	id	PK
uuid	inventory_id	FK
string	seat_code	
string	seat_class	
boolean	is_available	
integer	version	

BOOKINGS

uuid	id	PK
uuid	user_id	FK
uuid	inventory_id	FK
string	booking_reference	UK
string	status	
decimal	total_amount	
string	idempotency_key	UK
datetime	created_at	

PASSENGERS

uuid	id	PK
uuid	booking_id	FK
string	full_name	
integer	age	
string	gender	
string	assigned_seat	

PAYMENTS

uuid	id	PK
uuid	booking_id	FK
string	payment_gateway_ref	
decimal	amount	
string	currency	
string	status	
datetime	created_at	

OUTBOX_EVENTS

uuid	id	PK
string	aggregate_type	
string	aggregate_id	
string	event_type	
jsonb	payload	
boolean	processed	
datetime	created_at	

---

7. Section E: Viva Voce & Defense Strategy

When defending Moveo in your MCA final project viva, examiners will ask specific architectural questions. Here are the top questions and exact model answers:

Q1: "Why did you choose a Microservices architecture instead of a Monolith?"

> **Model Answer:** "A travel booking platform handles disparate workloads. Search queries have high read volume (100:1 read-to-write ratio), while booking and payments require strict consistency and high write transaction processing. With a monolith, scaling search scales the whole app. In Moveo, microservices allow independent horizontal scaling, isolated database-per-service performance, and fault isolation (e.g., if Recommendation fails, Booking remains 100% operational)."

Q2: "How do you handle Distributed Transactions across independent databases?"

> **Model Answer:** "In a microservices environment, traditional 2PC (Two-Phase Commit) locks databases across networks and does not scale. Moveo uses the **Saga Pattern (Orchestration)** managed by the Booking Service. If payment fails after reserving a seat, the Saga Orchestrator automatically fires a compensating transaction to release the seat in the Inventory service."

Q3: "What happens if the database updates but the event notification fails to publish to Kafka?"

> **Model Answer:** "We solved the Dual-Write problem using the **Transactional Outbox Pattern** (`backend/shared/patterns/outbox.py`). Instead of writing to Kafka directly inside the HTTP request, the DB transaction commits both the booking record and an event payload into an `outbox` table in the *same* local database transaction. A separate background worker reads from the `outbox` table and publishes to Kafka with at-least-once delivery guarantees."

Q4: "How do you prevent double-booking or double-payment if a user clicks 'Pay' twice?"

> **Model Answer:** "We implement **Idempotency middleware** using a unique `X-Idempotency-Key` header generated on the Next.js client for every booking attempt. The API Gateway stores this key in Redis SET-NX with a lock timeout. Duplicate requests with the same key are caught at the gateway and returned cached responses without re-executing payment or seat allocation."

Created for Moveo MCA Final Year Major Project — 2026/2027 Session.